数据库查询工具实战手册：从 Cursor 与 Spec Kit 到 PostgreSQL、MySQL

六讲实操 · 原提示词 · 模板

更新：2026-10-29

目录

1. 阅读说明
2. 三天建议时间与产物
3. 步骤 1 建立独立练习环境
4. 步骤 2 学会给 Cursor 提供范围与规则
5. 课程实验：把 Cursor、Rules 与上下文串起来
6. 步骤 3 从需求写出可验收的规格
7. 课程实验：按阶段实现，再用结果推动下一轮
8. 步骤 4 跑通连接与元数据
9. 步骤 5 实现并验证手写 SQL 查询
10. 步骤 6 把接口串成可操作的界面
11. 步骤 7 用元数据生成 SQL，并保留人工执行门槛
12. 步骤 8 增加 MySQL，并检视扩展架构

13. 步骤 9 测试、评审与打包复现说明
14. 六个课堂复现案例：怎样从现象走到下一步
15. 可填写模板与材料包
16. 扩展边界与排错索引
17. 作者原提示词全集
18. 课堂屏幕输入与未展示范围
19. 运行时提示词与消息构造原代码
20. 来源与逐字稿导航

阅读说明

> 手册类型：定制版

这本手册把 w2 的五讲、w3 的 MySQL 扩展课，以及同项目的提示词、规格、任务、评审报告和代码放到一条可执行路径里。目标是做出“添加演示数据库→看结构→手写查询→自然语言生成 SQL→人工检查后执行”的本地工具。课程中的做法按其当时上下文保留；源码的实际状态和助手新增的修补建议分别注明。

你可以先学会把需求交给 AI，再用课程项目练习完整交付。无需先通读全部框架文档，但每一阶段要保存能证明结果的文件或响应。生成了代码、任务打了勾、AI 说测试通过，都不能替代你本机的运行结果。

逐字稿是本地中文自动转录，提供原视频时间戳与 SRT，未逐字人工校订。技术名词、路径和命令以原始提示词、代码及截图为准；课堂口述不是聊天档案中缺失的原提示词。本手册不承诺复制讲者的速度，模型与工具的当前可用版本、订阅和费用需在实际使用时核对。模板中的“待填写”和演示库数据均为练习输入。

路线图

阶段	完成什么	跳转	留下的证据
准备	独立副本、环境、演示数据库	步骤1	版本清单、目录、安装日志
理解工具	Cursor 的上下文、Rules、分工	步骤2	一次限定范围的修改及差异
规格	constitution→spec→plan→tasks	步骤3	可核对的规格、接口与任务表
实现	本地 API、连接、元数据、手写查询	步骤4、步骤5	请求/响应、表结构、结果
体验	侧栏、编辑器、结果表格	步骤6	操作截图与错误状态
AI	元数据作为上下文生成 SQL	步骤7	生成 SQL 与独立执行结果
扩展	MySQL 方言、适配器与评审	步骤8	两类库同一验收表
交付	测试、修订、复现说明	步骤9	实际日志与未完成项

先不做公网部署、多租户、收费和接入真实业务库；以后再看扩展边界。完整原提示词、运行时提示词和模板随时可查。

本次条件

你的目标是把课程变成能照着做的详细手册，保留原提示词、真实截图和可编辑模板。用户已确认 w21 指 w2，按四个项目拆书。数据库项目在 w2 建立，在 w3 扩展 MySQL，因此合成一本；RaFlow 另成一本。

准备有终端与代码编辑器的电脑，以及可供练习的 PostgreSQL；MySQL 在扩展阶段再准备。Python 以课程 `pyproject.toml` 的 `>=3.12` 为准，前端按项目锁文件安装。没有真实 OpenAI key 也可先跑手写 SQL；源码导入阶段要求 key 存在，使用明确的假值只供离线检查，不能据此验收自然语言功能。[O1]

沿用你已给的条件：一人，周一至周三10:00–19:00，13:00–14:00午休，每天8小时。下表按24小时安排观看、操作和排错，是助手建议，不承诺在外部资源受阻时跑完全部功能。一次只推进一个阶段，阻断时记录停点。

条件不同时怎么调

- 时间更少：先做工具实验与规格，再做手写SQL；MySQL与架构重构后移。
- 缺真实key：继续配置、纯解析、模拟测试与手写查询；不能把假值验收成模型可用。
- 缺数据库：先做规格、SQL解析和隔离单元测试；等待演示库后补真实连接和执行记录。

三天建议时间与产物

六讲原视频合计约5小时25分钟；下表包含完整观看及暂停操作，合计24小时。讲者说“两三个小时”是现场开发自述，不是你的完成承诺。[O10]

日与时段	观看和动手	用时	当段产物
周一10–13	w2第1–3讲、规则/上下文实验	3小时	规则/命令、交接记录
周一14–16	w2第4讲、constitution/spec review	2小时	范围、PRD、接口数据模型
周一16–19	plan/tasks核对与环境准备	3小时	阶段验收矩阵、安装日志
周二10–13	w2第5讲前半、Phase1/2	3小时	连接/元数据/手写SQL证据
周二14–17	UI/白屏/样式回归	3小时	成功/空/加载/失败截图
周二17–19	第5讲后半、Phase3	2小时	NL需求、生成SQL和人工审查
周三10–13	w3 MySQL课、两类库回归	3小时	MySQL测试数据与同口径验收
周三14–17	analyze/架构/测试修订	3小时	报告、单项修复与回归
周三17–19	对照规格、复盘、剩余问题	2小时	真实结果表与下一步

这轮要留下什么

产物	数量口径	做到哪才计数
逐字稿	w2五讲＋w3一讲	带原时间戳的机器转录，非人工认证
规格链	constitution/spec/plan/tasks各一套	不拿任务勾选代替运行
查询场景	连接/结构/手写/NL各一条起	两种库分别记通过/失败/未测
原输入	文件10段＋已核屏幕补充	文件与视频类别分开，部分输入留缺口
模板与记录	每关键动作一份	〔〕改成自己真实条件

阻断缺口

- G1 当前副本后端单元测试64通过、20失败，前端锁文件冲突且后续安装因网络中止，不能宣称整体通过。
- G2 外部演示DB、真实模型生成与整本用户试用未验收；机器ASR和课程截图不能补成你的结果。
- G3 屏幕输入没有全部出现在聊天文件，补充台账逐条标完整/部分；不从口述补写看不见的尾句。

步骤 1 建立独立练习环境

先把安装、配置和演示数据隔离，再检查每个入口。这样报错时能区分课程本身的缺文件、锁文件冲突与本机数据库条件。

目标： 让每次安装、迁移与查询都作用于明确的练习目录和演示数据库。

1. 从课程项目复制 `w2/db_query` 到个人工作目录，保留原件，记录当前源码与锁文件版本；打开项目内 Makefile、后端 pyproject.toml 和前端 package.json。[O1]
2. 在后端先准备隔离的数据目录及环境变量，再安装依赖；前端单独按锁文件安装。把终端输出存日志。[O1]
3. 在独立演示库准备少量虚构数据，用只读账号连接查询工具；不要直接运行课程里会创建、删表或种子覆盖的数据库脚本到已有数据库。[O1]
4. 在两个终端分别启动后端与前端；用 `/health` 和页面能否打开分别判断，不用一个成功结果代替另一个。[O1]
5. 记录报错、命令、目录与退出码；遇到下面的已知安装缺口，先做最小修补，再继续下一步。[O1]

> **做完的标准**： 明确知道本地 SQLite 文件在哪里、目标演示库是什么；两个服务分别有启动证据，尚未验证的外部服务写清楚。

- [ ] 记录两项端口归属、实际服务目录及健康检查响应。
- [ ] 把安装失败或成功日志、练习数据路径写入 T01。

从正确目录执行

先查端口归属，避免把其他项目响应认成本项目。制作时本机的 8000 端口由另一个 video-mcp 服务占用，未停止它。读者可用 `lsof -nP -iTCP:8000 -sTCP:LISTEN` 和 `lsof -nP -iTCP:5173 -sTCP:LISTEN` 确认进程；若后端改成 8001，需同时修改前端私有环境配置 `VITE_API_BASE_URL=http://127.0.0.1:8001`，重启 Vite，并把下列 curl 改成相同端口。网页打开只能证明前端服务响应，不能代替本项目 `/health`。[O1]

以下是助手根据源码组织的命令；工作目录先进入你的练习副本 `db_query/backend`。只做离线检查时使用假 key；真正生成 SQL 前在本机私有配置里换成实际 key。

```
export DB_QUERY_DATA_DIR="$PWD/.practice-data"
export OPENAI_API_KEY='test-not-real'
uv sync --frozen --extra dev
uv run alembic upgrade head
uv run uvicorn app.main:app --host 127.0.0.1 --
port 8000
```

另开一个终端，进入同一副本的 `frontend`：

```
npm ci
npm run dev -- --host 127.0.0.1
```

再开终端验证：

```
curl --fail-with-body http://127.0.0.1:8000/health
```

原 Makefile 的 `make dev` 把两个前台常驻服务列成顺序依赖，第一个服务不退出时第二个不会按注释所说自动并行。初学时用两个终端最容易分辨哪边失败。`make setup` 包含安装和迁移，却在末尾才提示配置 key；应把配置提前。默认 SQLite 是 `~/.db_query/db_query.db`，不要把它与你的练习文件混用。[O1]

已知安装缺口与修补范围

当前材料的后端 `pyproject.toml` 声明 `readme = "README.md"`，而 `backend/README.md` 不存在。独立副本实际安装复现了 `OSError: Readme file does not exist: README.md`。助手仅在练习副本加了明确标注的 README 占位说明，使打包元数据可读取；这不等于应用功能全部通过。也可以在自己的副本把 readme 指向真实课程说明，但需要相应维护锁文件。前端还实际复现 `npm ci` 因 package.json 与 package-lock.json 不一致退出，包含 React 18/19 差异；先保留旧锁文件，在副本修正依赖与锁文件，再重新构建，不能跳过错误宣称安装成功。[O9]

原提示词与本步模板

原输入 W2-P04 直接要求修复 `make setup` 并通过前后端测试，没有提供故障复现细节。本步可用下面的助手模板补足诊断输入：

工作目录：〔练习副本绝对路径〕
目标：只修复安装/启动，暂不改产品功能。
原命令与完整报错：〔逐字粘贴〕
请先检查 `pyproject`、`package`、锁文件、`Makefile` 和环境变量读取顺序，
说明根因及最小改动；列出改动文件、复跑命令、退出码和仍未验证的部分。
不要读取或打印真实 `.env`，不连接我的已有业务数据库。

保存版本见 `templates/T01-environment-and-debug.md`。卡住时先问“是在解析配置、安装包、迁移还是启动服务”，不要把四类问题混成一条报错。

步骤 2 学会给 Cursor 提供范围与规则

规则与上下文决定AI看见什么、受到什么约束。先用一个小改动练习，再把同样方法用到数据库工具。

目标： 用一次小修改练习“明确上下文→限定改动→检查差异”的协作方式。

1. 在 Cursor 打开练习项目根目录，先让 AI 解释相关目录和入口，给出你要改的文件引用，不把整个磁盘都作为任务范围。[O2]
2. 阅读已有 `.cursor/rules/`；把 Python/FastAPI、类型、数据模型与项目样式分别放到适用规则里，检查规则是否会匹配本项目。[O2]

3. 选一个不涉及真实数据库的小任务，例如改一个错误提示，说明输入、预期、不得改动的部分，并提供对应组件或路由。[O2]
4. 看 AI 提议的文件差异，拒绝无关依赖升级和大面积格式变更；运行该范围的检查，再接受修改。[O2]
5. 把发现的项目固定约束加入项目规则，把这次临时任务留在聊天里，避免规则越积越杂。[O2、O3]

> **做完的标准**： 能说出这次 AI 看到了哪些文件、受哪些规则约束、改了什么，以及你用什么证据接受它。

- [ ] 在 T02 写清本次引用文件、规则与禁止改动范围。
- [ ] 对一条 AI 修改逐项核对规则和差异。

Rules 应该放什么

课程文件有 `python-fastapi-backend.mdc`、`specify-rules.mdc`、`rust-best-practices.mdc` 以及样式参考 `style.md`。不要因文件存在就认为每次任务都加载了它；`.mdc` 的头部配置和匹配范围要与项目路径对应。Rust 规范对本项目 Python/TS 业务无直接必要，不宜当作所有文件的强制规则。原文件完整随模板包保留。[O3]

本项目的稳定约束来自 W2-P01：后端 Ergonomic Python、前端 TypeScript、严格类型、Pydantic、输出 camelCase，以及原型不做认证。最后一点只说明课程范围，不能据此直接公开无认证的数据库工具。

把约束变成可检查条件

约束	你实际检查什么	容易误判的地方
Python/TS 类型	函数输入输出、API DTO、前端结果结构	文件后缀是 TS 不代表没有 any
Pydantic	API 输入输出模型与边界值	只在数据库实体上加模型不等于接口一致
camelCase	JSON 中 `rowCount`、`dataType` 等字段	内部 snake_case 可以保留，不要盲目全局替换
KISS/YAGNI	当前验收需要哪些功能	不为未来所有数据库先写通用平台
规则范围	哪些路径匹配、何时引用	把样式参考误当能自动生效的全局规则

原提示词与本步模板

原文见 W2-P01。下面是助手给小任务用的限定模板，并不是课程原话：

> 请先阅读〔相关路径〕及本目录适用规则。
> 这次只完成：〔一个可观察行为〕。
> 输入：〔请求/页面状态〕；预期：〔具体结果〕。
> 保持：〔接口字段、既有功能〕；不涉及：〔升级/重构范围〕。
> 先说明计划和需读取的文件，再给最小`diff`；完成后提供实际检查输出。

保存为 `T02-cursor-task.md`。当一次对话开始讨论新目标或上下文过长时，先把已确认决策落到规格/任务文件，再开启下一轮，不让关键条件只留在聊天记忆中。

课程实验：把 Cursor、Rules 与上下文串起来

这部分对应前3讲的工具操作，先拿一个小改动练习，再进入数据库主线。屏幕中的模型、菜单和订阅是录课时状态；不要求为了复刻课程购买相同套餐。

模式与任务怎么搭配

课程里的入口	适合用来练什么	验证结果
Tab补全	重复结构、同类文字/代码修改	接受前看建议是否改错其他位置
Inline Edit（课中 Command+K）	选中一小段代码给局部指令	差异限定在选区及必要依赖
Agent对话	读文件、计划、跨文件实现与测试	检查文件范围和实际工具结果
Rules & Memories	语言偏好、项目约束、按需规则	确认当前任务确实引用了所需规则
Tools & MCP	浏览器检查、终端/控制台上下文	插件真实可调用，不只配置了名字
Docs	引入相关文档作为上下文	使用时显式引用，并核版本
模型选择	复杂规划与具体实现分别尝试	以任务质量、延迟和实际成本选择

课程第1讲展示登录后在 Settings 看账户、布局、模型、工具/MCP、Rules、索引和文档。它不是一份对当前订阅与隐私条款的核验；按你正在使用的版本读实际设置。讲者建议用较强模型做规格与计划，用较快模型编码；这是工作分工经验，不是模型排名结论。[O2]

工具循环可以按"输入目标和相关文件→模型决定下一动作→读取/修改/执行工具→工具结果回到上下文→继续判断"理解。系统提示词、用户输入、规则、工具返回最终都影响模型输入。上下文越多不等于越好：陈旧错误和无关规则也会占空间或造成冲突。

做一条项目规则

先打开 `.cursor/rules/python-fastapi-backend.mdc` 看 front matter，再读正文。`description` 说明用途，路径匹配与 `alwaysApply` 等设置决定是否自动带入；也可按任务手动引用。全局用户偏好适合放沟通语言、shell等条件，项目架构和API约定放版本可管理的项目规则。

主题尽量单一，给正面动作和代码例子，避免只写"高质量""不要乱改"等难验收的要求。新增规则后做一次匹配文件的小任务，查看AI实际产物；再用不相关文件检查是否误触发。规则内容过期时按当前代码修订，并保存变化理由。

课程第1讲用 Rust 规范演示"让AI先读已有规则再生成新规则"，包括应用错误用 anyhow、库用 thiserror、并发优先 channel、Tokio、Axum和标准转换 trait。这是工具实验，不要求数据库项目增加 Rust。原 `.mdc` 完整放在模板包；视频中的输入草稿若只显示部分，按截图的可见范围保留，不补写成完整原话。

做一个可重复命令

课程还演示创建图片优化命令、commit命令和create-pr命令。命令文件完整保留在 `cursor-original-commands/`，其中图片优化涉及扫描、统计、备份、工具选择和处理结果；这属于有实际文件影响的操作，应先限定练习目录及备份，不直接对课程原件运行。

练习可按下面顺序：

1. 打开 `optimize-images.md`，读它会扫描什么、调用什么工具、怎样备份，不立即执行。 [O10]
2. 在临时目录放两张可替换的练习图片，给命令限定该目录。 [O10]
3. 对比处理前后文件名、尺寸、大小、画质和备份；一个指标变好不代表其余都合格。 [O10]
4. 查看commit命令如何读git状态、组织message；自己审查暂存范围再提交。公开PR是另一项动作，不由本练习自动触发。 [O10]

助手模板 `T11-rule-command-lab.md` 提供规则范围/触发、正反例、命令输入输出、副作用和验证记录。

设计风格规则：完整保留课堂的纠偏过程

第2讲从 MotherDuck 风格研究开始，要求提取 border、padding、margin、font、color、typography，再把结果写成前端风格规则。讲者分别比较多个工具的回答，选结果粘入规则，然后让AI按规则改课程网站。这是同周工具演示，改的是 `site/`，不是凭空新增数据库需求。

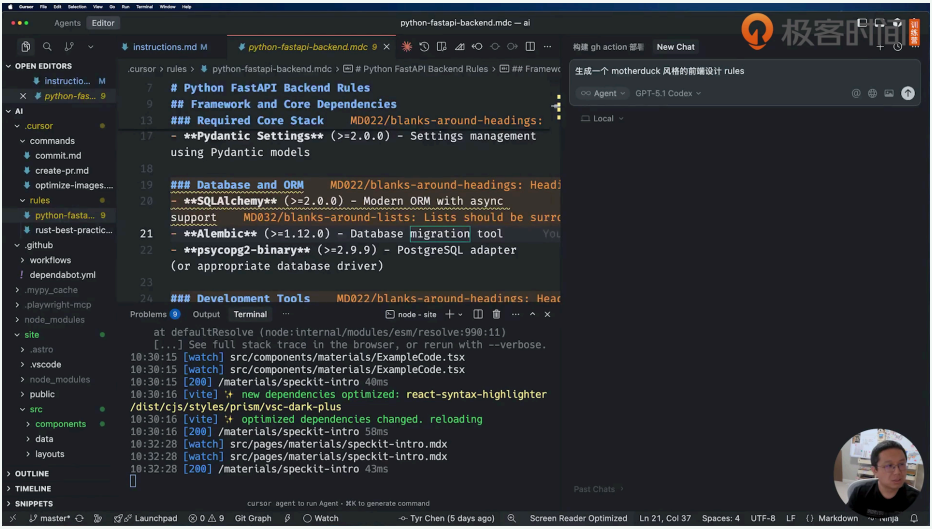
过程里出现了不合意的圆角、边框、背景、页面间距、Footer和按钮等问题。讲者进一步要求直角、黑粗线、样式文件统一、用Playwright逐页检查，再继续迭代。这说明"抽设计规则→整站应用→看实际页面→修正规则和界面"要闭环；一个规则文件写好了，并不证明它准确复现参考网站。

把它迁移到数据库工具时，先保存自己的界面基线与目标布局，再提供参考图和 `style.md`。不要把网站的整站重构指令原样套到数据库目录。助手模板 `T12-style-extraction.md` 要填写实际页面、参考截图、设计要素、禁止误改区域、四态检查与验收证据。

屏幕原输入 DB-V01 | 生成MotherDuck规则

第2讲00:32:35，尚未附加后续参考要素，不把后续内容并进这句。状态：完整可见，不是从ASR拼出的逐字全文。[O11]

```
生成一个 motherduck 风格的前端设计 rules
```

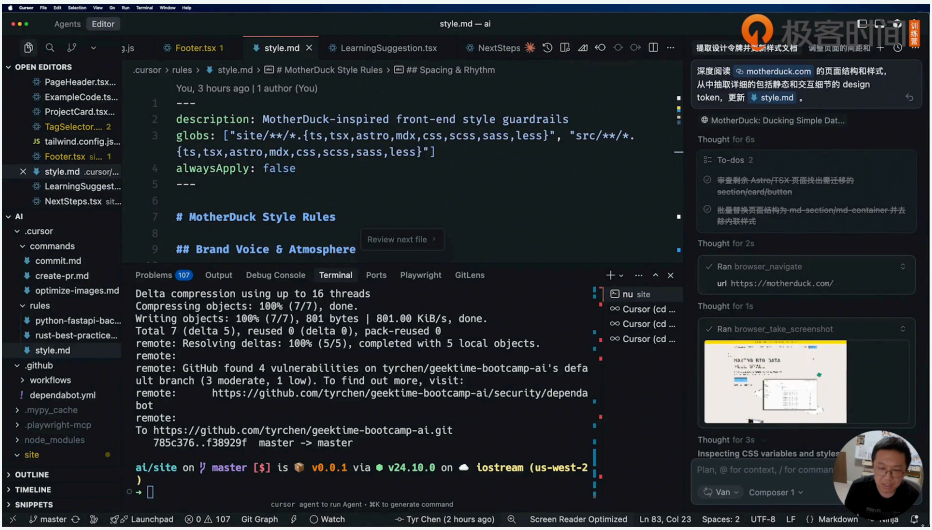


图注：真实课程视频帧，第2讲00:32:35，尚未附加后续参考要素，不把后续内容并进这句；2026-10-29提取/核看，保留录课水印，不是本次实机结果。

屏幕原输入 DB-V02｜继续提取设计token

第3讲00:07:03可见上次消息回顾；链接/附件标签转为文件名，软折行合并。状态：完整可见，不是从ASR拼出的逐字全文。[O11]

> 深度阅读 `motherduck.com` 的页面结构和样式，从中抽取详细的包括静态和交互细节的 `design token`，更新 `style.md`。



图注：真实课程视频帧，第3讲00:07:03可见上次消息回顾；链接/附件标签转为文件名，软折行合并；2026-10-29提取/核看，保留录课水印，不是本次实机结果。

Memory Bank、Spec Kit 与持久化上下文

第3讲先介绍 Memory Bank 的模式、文档与规划，讲者当时认为已不必额外使用；这不等于该工具今天不可用。保留其启发：把项目背景、当前状态、下一步与关键决策落到文件，下轮任务只引入需要部分。

接着引入Spec Kit：CLI初始化后生成 `.specify` 和对应编辑器命令，再以 constitution→specify→plan→tasks→implement 推进；clarify、analyze、checklist等用于澄清或一致性检查。使用本次仓库已附命令时先读脚本是否创建分支、怎样定位feature目录，避免在多项目仓库写错位置。不要不加检查照搬历史安装命令或把现有目录重新初始化。

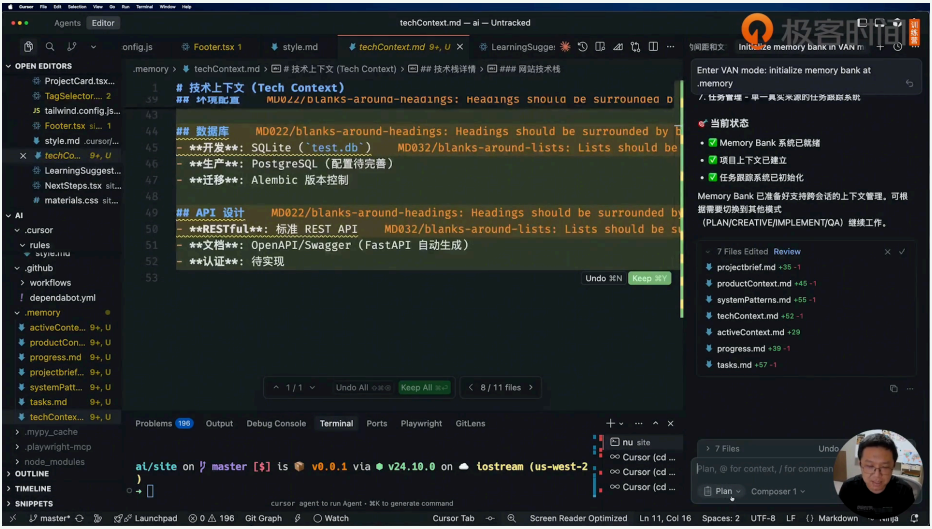
时点	放进持久文档什么	下一轮提供什么
想法	用户问题、范围、非目标	原始需求与项目原则
计划	技术约束、接口、数据、分期	已确认规格与待实现阶段
实现后	实际完成、检查结果、缺口	当前代码路径、日志和下一任务
出错时	完整报错、尝试和结果	最小复现，不堆全部历史对话
变更时	新需求、受影响契约和任务	新旧差异及回归范围

第5讲在Cursor额度用尽后切到Claude Code，继续读取同一批规格和任务，体现文件化上下文能支持工具切换。手册不要求复制讲者的权限模式，也不把其 `yolo` /bypass展示当成必要步骤。助手模板 `T13-context-handoff.md` 可用于不同工具间交接。

屏幕原输入 DB-V03 | Memory Bank初始化

第3讲00:09:50，历史工具实验，不要求数据库项目安装它。状态：完整可见，不是从ASR拼出的逐字全文。[O11]

```
Enter VAN mode: initialize memory bank at .memory
```



图注：真实课程视频帧，第3讲00:09:50，历史工具实验，不要求数据库项目安装它；2026-10-29提取/核看，保留录课水印，不是本次实机结果。

步骤 3 从需求写出可验收的规格

先把需求、技术选择和执行计划分开写，避免一句“做个查询工具”让代理同时猜三层意思。每份产物必须能传给下一阶段。

目标： 把“数据库查询工具”拆成有接口、数据和验收条件的实现任务。

1. 用 W2-P01 明确项目原则，再把 W2-P02 的完整基本想法作为需求输入，先核对要做和暂不做的功能。[O4]

2. 参照仓库已有 `speckit.constitution`、`speckit.specify`、`speckit.plan`、`speckit.tasks` 命令文件推进；先查命令实际存在及输出路径，再运行，避免历史路径写到别的周次。[O4]
3. 检查生成的 spec、research、data-model、contracts/api-v1.yaml、plan、tasks 与 quickstart：需求有没有对应接口，接口有没有数据模型，任务有没有验证方式。[O4]
4. 把任务按基础设施、核心连接/查询、自然语言、体验与测试依赖拆批；每批通过一个完整使用场景后再进入下一批。[O4]
5. 对照源码修正规格中的版本、路径和实现状态，不把历史文档勾选与评审分数当作本次验收。[O4]

> **做完的标准**： 手上有"需求→接口→模型→文件→验证"的对应表，且你能指出第一批实现后用户究竟能完成什么。

- ☐ 把一个需求映射到接口、模型、任务与验收样例。
- ☐ 在 T03 标出三阶段范围及当前停点。

课程已经留下的文档

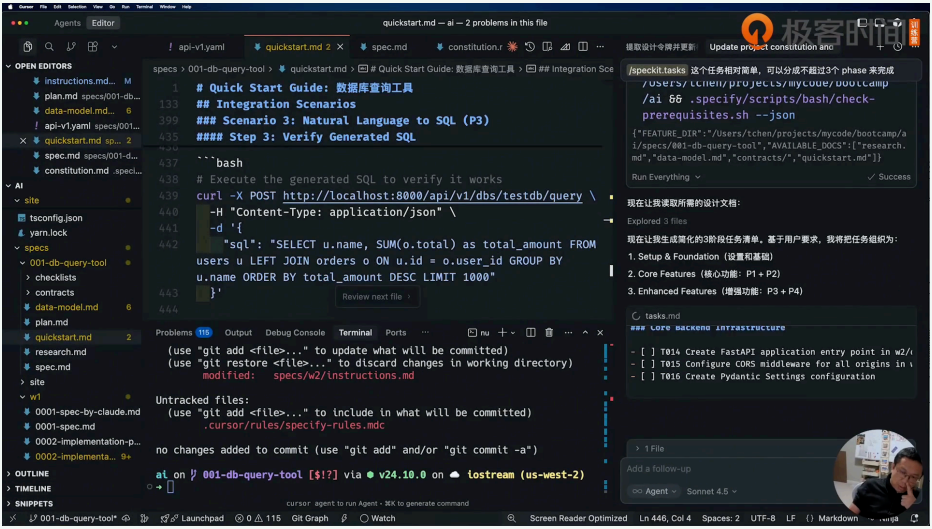
实际目录是 `w2/specs/001-db-query-tool/`；某些正文仍写 `/specs/w2/001-db-query-tool/`，使用前换成你的真实路径。文档按原样附在 `templates/course-spec-kit-output/`。仓库里的命令模板和生成产物不同：命令模板告诉 AI 怎样工作，spec/plan/tasks 是某次工作的结果，不要互相冒充。

文件	该回答的问题	你补的验收
spec.md	用户做什么、功能范围	添加连接后能看到什么
research.md	技术选择与约束	锁文件与代码是否仍一致
data-model.md	连接、元数据、历史如何关联	名称重复、删除、过期如何处理
api-v1.yaml	请求/响应契约	真正路由/字段/状态码是否一致
plan.md	实现架构和顺序	关键依赖与环境前提
tasks.md	具体文件与任务	勾选项是否有测试日志
quickstart.md	从新环境怎样跑起来	两终端、key提前、缺README处理

屏幕原输入 DB-V04｜限定任务阶段

第4讲00:39:30；输入完整，代理任务说明与总数是另一部分。状态：完整可见，不是从ASR拼出的逐字全文。[O11]

```
/speckit.tasks 这个任务相对简单，可以分成不超过3个 phase 来完成
```



图注：真实课程视频帧，第4讲00:39:30；输入完整，代理任务说明与总数是另一部分；2026-10-29提取/核看，保留录课水印，不是本次实机结果。

最小需求分解

- 连接管理：输入名字和 URL，验证连接，存 SQLite，列出和删除。
- 元数据：按数据库类型读表、视图、列、主键等，缓存到 SQLite，提供刷新。
- 手写查询：解析 SQL，限定语句范围，无外层 LIMIT 时补上默认上限，返回列和行。
- 自然语言：把已获取的结构作为上下文，请模型生成 SQL，用户检查后再执行。
- 界面：连接侧栏、结构树、Monaco 编辑器、结果表格、错误/加载/空态。

原始需求提到用 LLM 把数据库信息转换成 JSON；现码由 Python 直接组织元数据 JSON，LLM 用于后续 SQL 生成。不要额外增加一次不必要的模型调用来“补齐”历史描述。[O1、O5]

原提示词与本步模板

完整 W2-P02 包括全部 API 示例。助手模板在 `T03-spec-to-tasks.md`：

> 原始需求：〔粘贴完整需求，不删边界条件〕
> 项目原则：〔类型、命名、认证范围、数据存放〕
> 请输出：用户场景、非目标、数据模型、API契约、失败行为、分阶段任务和验收矩阵。
> 每个任务给实际文件路径、前置依赖、可观察完成标准。
> 遇到不明确条件先集中列问题；不能把假设当成我已经确认。
> 完成后列需求覆盖表，指出仍没有对应任务的需求。

课程实验：按阶段实现，再用结果推动下一轮

第5讲并非“一条implement后全成功”。它经历安装、版本、框架兼容、界面与运行时测试的多次失败，再转到Claude Code继续同一套规格。按下面顺序做，才能复现讲者真正的工作方式。[O10]

阶段一：先让框架、配置、迁移与健康检查成立

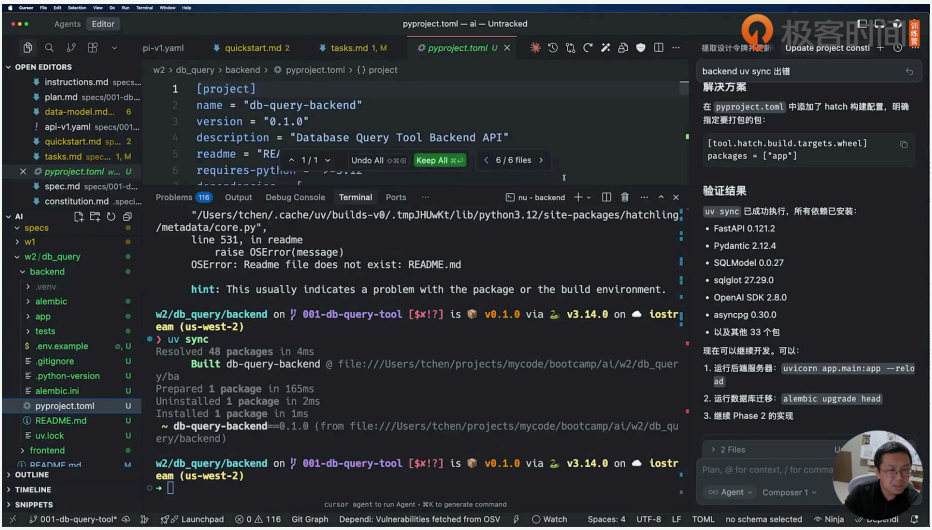
1. 打开tasks.md的Phase1，列出后端/前端目录、依赖、配置、SQLite模型/迁移和健康检查，确认任务文件属于db_query而非同仓库别的项目。 [O10]
2. 在工具中引用当前tasks和plan，限定这轮只实现Phase1。课程屏幕确实输入implement，但某帧底部后缀被遮住，完整句子未核；不要把下方助手模板称为原文。 [O10]
3. 执行uv sync，先读真实错误。课程00:03:10画面出现`Readme file does not exist: README.md`，之后显示重新安装；本次独立副本也遇到同类缺件。修包元数据与升级全部依赖是两种操作，不要混做后就失去根因。 [O10]
4. 为实际完成的API生成REST请求，再用curl逐个测。课程00:06:00–00:08:30曾打开别的项目请求文件，出现“标签”接口，讲者承认拿错文件；先核health和dbs路径，不能见404就改后端。 [O10]
5. 保存这一阶段的命令/工作目录/响应，再核任务勾选。框架能启动后才进入连接查询，不能用代码行数替代用户场景。 [O10]

本节动作均对照第5讲00:00:00–00:10:00及源码Makefile/fixtures。[O10、O1]

屏幕原输入 DB-V05｜uv安装失败反馈

第5讲00:03:10，屏幕同时显示缺README错误与后续重跑，不能把代理成功宣告当本次安装结果。状态：完整可见，不是从ASR拼出的逐字全文。[O11]

backend uv sync 出错

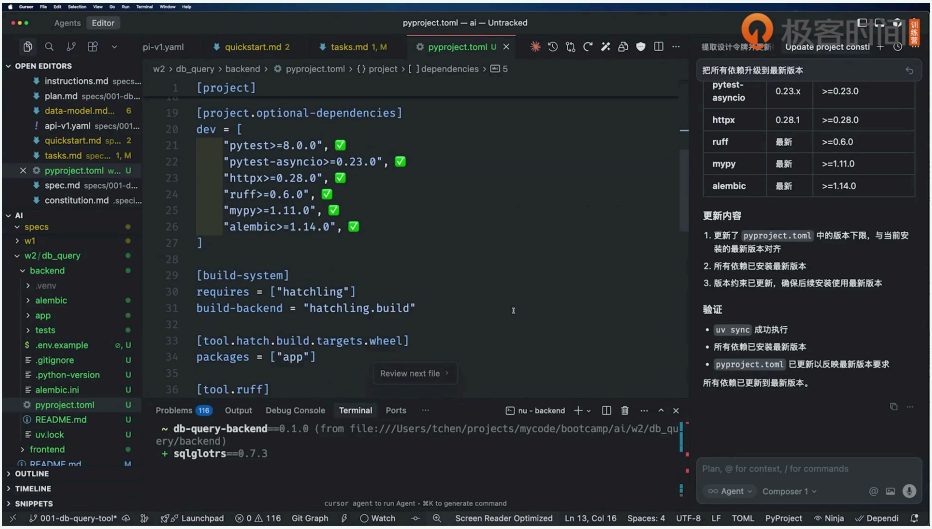


图注：真实课程视频帧，第5讲00:03:10，屏幕同时显示缺README错误与后续重跑，不能把代理成功宣告当本次安装结果；2026-10-29提取/核看，保留录课水印，不是本次实机结果。

屏幕原输入 DB-V06｜升级依赖

第5讲00:04:30；这是课程输入，复现现成代码不照此盲目升级。状态：完整可见，不是从ASR拼出的逐字全文。[O11]

把所有依赖升级到最新版本



图注：真实课程视频帧，第5讲00:04:30；这是课程输入，复现现成代码不照此盲目升级；2026-10-29提取/核看，保留录课水印，不是本次实机结果。

阶段二：连接、结构、手写SQL与UI一起核

1. 用完整DB-V07实现Phase2，再读实际路由和schema。至少添加一个演示连接，取结构，执行一条小SELECT；本手册步骤4–6提供具体请求和界面检查。 [O10]
2. 启动两个服务，用本项目fixtures/test.rest与curl测试已实现路由，再用Playwright/实际浏览器打开同一场景。代理报“所有任务完成”后仍要自己点添加、刷新与执行。 [O10]
3. 若页面白屏，分别收集console错误、网络失败与依赖差异；课程00:29:00–00:34:00遇到React/组件兼容并尝试退到18.3.1，不表示所有人都必须降级。先比较你的package和lock，再做最小改动。 [O10]

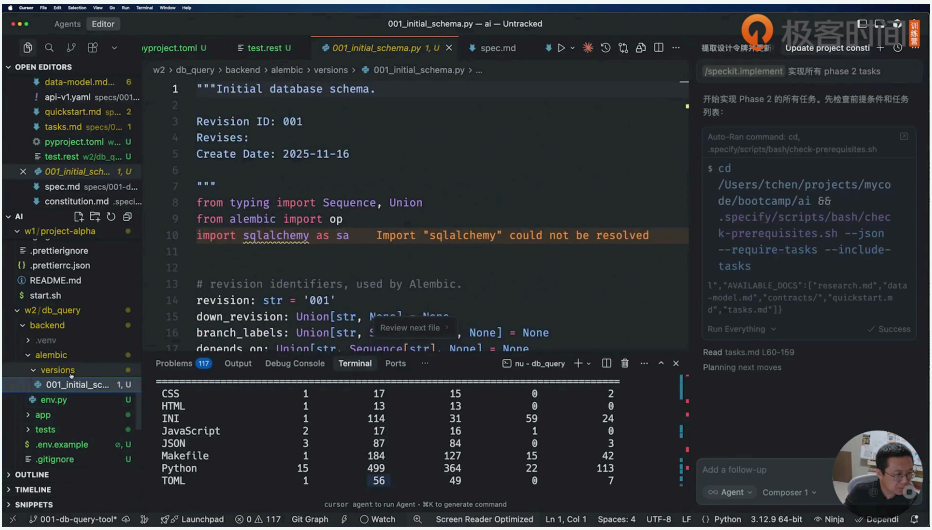
4. 给UI实际参考截图和一张区域关系图：左侧连接列表/结构树，右侧编辑器/结果；再指定token、可搜索、默认展开、固定工具栏与可滚动区域。课程00:49:00–01:01:00持续修高度/边框/对齐，说明“看起来差不多”不是行为验收。 [O10]
5. 记录切库、失败和空结果后表格是否仍显示上一次数据。画面有表格并不能证明当前输入已执行，尤其新SQL尚在编辑时。 [O10]

这些是第5讲00:10:00–01:02:00的操作链，配合步骤4–6逐项执行。 [O10、O6]

屏幕原输入 DB-V07｜实现第二阶段

第5讲00:10:50，保留大小写与phase任务范围。状态：完整可见，不是从ASR拼出的逐字全文。[O11]

```
/speckit.implement 实现所有 phase 2 tasks
```



图注：真实课程视频帧，第5讲00:10:50，保留大小写与phase任务范围；2026-10-29提取/核看，保留录课水印，不是本次实机结果。

工具切换：Cursor额度用尽后怎样继续

课程00:18:00–00:21:30因Cursor额度改用Claude Code，给同一项目路径、fixtures/test.rest和原测试要求。你要交接的是规格、已完成阶段、实际失败日志和下一步，而不是“把整段旧聊天再粘一次”。先列当前plan/tasks/源码路径，核新工具确实读取了正确feature目录，再继续原阶段。MCP浏览器工具按自己现有授权配置；录课的bypass/yolo不是复现必须条件。[O10]

阶段三：在手写查询闭环后加入自然语言

1. 将Phase2可用事实写到任务记录，接着只实施Phase3的元数据上下文、模型服务、接口、前端输入与错误展示。 [O10]

2. 先核模型配置、前后端返回字段，再验证生成后是否回填编辑器、由用户主动执行。课程01:31:00–01:34:00连续生成报错，讲者要求检查类型/调用，直到能看到SQL；不要把“key有效”当接口契约已对。 [O10]
3. 选结构更复杂的演示库，要求跨表、过滤和排序，核schema前缀。课程换过todo、Lego、员工库；空结果可能是数据或条件，不自动判模型坏。 [O10]
4. 比较问题本意、生成SQL与结果。课程01:38:00–01:40:00薪水排行出现重复人，讲者明确看到重复；这只证明查出了数据，不能证明“每人一条/当前工资”等业务语义正确。 [O10]
5. 把错例连同完整需求、表结构与实际SQL保存，用T17澄清口径后再生成。使用真实key前确认外部调用条件；本次未调用模型。 [O10]

本节依据第5讲01:02:00–01:41:00。[O10、O7]

分阶段推进可套用模板 T14（助手补写）

```
当前项目：〔绝对路径〕；规格/plan/tasks：〔实际文件〕。
这一轮只实现〔Phase/任务范围〕，前置验收证据：〔日志/响应〕。
请先核当前代码与已完成任务，不重做其他项目。
完成后分别给：实际diff、运行命令/工作目录、健康检查、一个用户场景、失败与未测项。
如果遇安装/白屏/类型错误，先保留完整报错并定位根因，再做最小修补。
禁止用任务勾选或生成总结替代实际验证；下一阶段先等本阶段结果可解释。
```

步骤 4 跑通连接与元数据

连接和元数据是不依赖模型的第一条真实回路。先证明数据库名字、结构与缓存对应，再谈自然语言。

目标： 先证明普通 API 能连接演示库并读出真实结构，再接模型。

1. 确认后端 `/health` 正常，用列表 API 检查当前练习 SQLite 中已有连接，不把目标业务库和配置库混淆。[O5]
2. 使用你准备的演示库只读连接，通过 PUT 添加连接；名称用字母、数字、连字符或下划线，保存响应。[O5]
3. 请求该名字的元数据，检查表/视图/列与数据库自身查询结果是否相符；结构改变后调用刷新再核对。[O5]
4. 分别试一次坏 URL、重名更新和不存在的连接，记录实际状态码；不要只做成功路径。[O5]
5. 将连接、元数据、截图和错误日志填入本步模板，合格后再做 SQL 查询。[O1、O5]

> **做完的标准**： 能从 API 返回中指出一个演示表的 schema、列、类型和主键，刷新后能看到结构变化；没有把真实密码写入记录。

- ☐ 保存一个表的列、类型和主键响应。
- ☐ 记录刷新前后变化或当前连接阻塞。

最小请求顺序

把下列虚构连接串改为你的演示配置；示例不创建数据库，也不授予权限。密码里特殊字符需正确编码，结果中含连接 URL 的部分保存前脱敏。

```
curl --fail-with-body
http://127.0.0.1:8000/api/v1/dbs
curl --fail-with-body -X PUT
http://127.0.0.1:8000/api/v1/dbs/demo_pg \
  -H 'Content-Type: application/json' \
  -d
'{"url":"postgresql://demo_reader:DEMO_PASSWORD@12
7.0.0.1:5432/demo_db"}'
curl --fail-with-body
http://127.0.0.1:8000/api/v1/dbs/demo_pg
curl --fail-with-body -X POST
http://127.0.0.1:8000/api/v1/dbs/demo_pg/refresh
```

现码会在 GET 元数据时使用缓存或拉取，POST refresh 强制更新；自然语言接口只读有效缓存，不会替你自动补齐元数据。缓存是否过期由模型里的 24 小时判断；配置项存在不等于所有调用都读取该配置。[O5]

数据分别存在哪里

内容	位置	作用
用户要查询的数据	PostgreSQL/MySQL 演示库	执行 SELECT 的目标
名称、URL、类型、状态	本地 SQLite 的连接表	下次复用配置
表/视图/列信息 JSON	本地 SQLite 的元数据表	结构展示与 LLM 上下文
查询和成功/失败记录	本地 SQLite 的历史表	排错与回看，源码按每库保留50条
API key	私有环境配置	后端请求模型，不应出现在前端包

连接字符串含敏感信息，SQLite 不是自动加密的保险箱。本地练习之外需要重新设计密钥管理与访问控制；这属于扩展边界。

原提示词与本步模板

原始 API 输入都在 W2-P02，课程 REST 请求保留在项目 `fixtures/test.rest`。助手记录模板 `T04-connection-metadata.md` 含：连接别名、类型、脱敏 URL、预期表、返回表、刷新前后差异、HTTP 状态、失败原因与下一动作。相同请求用 curl 成功后再检查前端，能把后端连接失败与页面展示失败分开。

步骤 5 实现并验证手写 SQL 查询

解析、授权和执行是不同边界。先用小查询确认返回形状，再用纯解析样例查限制，避免拿“只读”一句话代替验证。

目标： 验证 SQL 经过后端检查、使用正确方言执行，并返回可理解的结果。

1. 从确认存在的演示表写一个显式列名、少量结果的 SELECT，先通过 API 请求执行，保存 columns、rows、rowCount 与执行耗时。[O5]
2. 阅读 SQL validator，分别验证没有 LIMIT、已有 LIMIT、错误语法及写语句的处理；这些失败用解析层测试，别向真实库发送破坏性语句。[O5]
3. 检查外层/内层 LIMIT、多语句、CTE、UNION 等边界，不把“根节点是 SELECT”当作完整只读保证。[O5]
4. 查询不存在表、空结果和带 NULL 的结果，确认错误信息与前端不会把失败当成上次成功数据。[O5]
5. 核对查询历史，区分输入 SQL、实际执行 SQL 和返回展示 SQL；当前包装器存在记录原输入而非补 LIMIT 后 SQL 的差异。[O5]

> **做完的标准**： 基础成功/失败样例都有响应证据，解析器限制及尚未修复边界明确写进验收表。

- [ ] 按 T05 留下正常查询、拒绝写入及多语句边界的实际结果。
- [ ] 分开记录解析通过与真实数据库执行通过。

请求例子

以下仅是虚构演示表；先替换为元数据确认存在的表与列。

```
curl --fail-with-body -X POST
http://127.0.0.1:8000/api/v1/dbs/demo_pg/query \
  -H 'Content-Type: application/json' \
  -d '{"sql":"SELECT id, title FROM
public.demo_items LIMIT 10"}'
```

后端现有路径是 `queries.py→execute_query_with_service→DatabaseService→适配器`。旧 `services/query.py` 仍存在，不能只看旧文件就判断新路由的实际行为。PostgreSQL 使用 asyncpg，MySQL 使用 aiomysql；统一结果包含列、行数和数据行。[O5]

不能跳过的边界

样例	应关心什么	当前源码线索
`SELECT 1`	自动添加外层上限	默认limit1000
`SELECT 1 LIMIT 5`	保留用户已有上限	不代表会把大于1000的显式值截断
`SELECT * FROM (SELECT 1 LIMIT 1) x`	内层上限不能代替外层	`parsed.find(exp.Limit)`会找到任意后代
两个分号分隔语句	必须只有一条真正可执行语句	`parse_one`的具体行为要实测，不能假定
`SELECT ... UNION SELECT ...`	根节点可能不是Select	当前validator可能拒绝此类合法查询
带写入CTE或有副作用函数	SQL形状不等于权限	还需要数据库只读权限及事务/超时策略

本次已在独立环境做7例**纯解析检查**，结果保存于`edit/practice/sql-boundary-results.json`：`SELECT 1`变成`SELECT 1 LIMIT 1000`；`LIMIT 5`保留；仅内层LIMIT时未补外层；`SELECT 1; SELECT 2`被校验接受但转换只留下第一条并加LIMIT；UNION被拒绝；只读CTE通过且补外层LIMIT；DELETE被拒绝。这没有连接数据库，没有执行破坏性语句，也没有证明只读授权或实际驱动行为。[O9]

因此把"默认加LIMIT"改写成明确有条件的实现现状：它用`parsed.find(exp.Limit)`查任意后代，内层LIMIT会被当作已有上限；多语句可能被静默截断，不能称输入被完整执行或被严格拒绝。原型LIMIT也不保证计算成本低；COUNT全表和复杂联表仍可能昂贵。

原提示词与本步模板

原始 SQL 条件在 W2-P02。助手验收模板`T05-sql-boundaries.md`要求：

> 检查目标：app/services/sql_validator.py 以及实际调用它的查询路径。
> 请用纯解析器/模拟适配器验证，不连接真实数据库。
> 覆盖：空串、语法错、SELECT、写语句、多语句、CTE、UNION、内外层LIMIT。
> 对每例记录原SQL、AST根类型、是否通过、转换SQL、预期及理由。
> 若修补，只改解析/验证范围；不要声称SQL解析本身等于数据库只读授权。

步骤 6 把接口串成可操作的界面

界面要让当前数据库、SQL和结果归属清楚。课程在这里反复调整布局，也暴露了样式与兼容性不能只看一次成功画面。

目标： 让用户明确知道选的是哪个库、编辑的是哪条 SQL、看到的是哪次结果。

1. 从页面左侧连接列表选择演示库，确认结构树和当前库标题同时变化；没有选择时应该显示空态。[O6]
2. 展开表和列，使用搜索定位结构，再把明确的表/列写进 Monaco 编辑器；手工核对方言、schema 与标识符引号。[O6]
3. 执行一条 SELECT，检查列头、NULL、长内容、分页与结果行数；切库或执行失败后避免误读旧结果。[O6]
4. 分别体验加载、连接失败、元数据失败、空结果、生成失败；记录操作、画面和接口响应的对应关系。[O6]
5. 使用 CSV/JSON 导出时核对文件内容与本次查询结果；保存一条从选库到结果的完整操作记录。[O6]

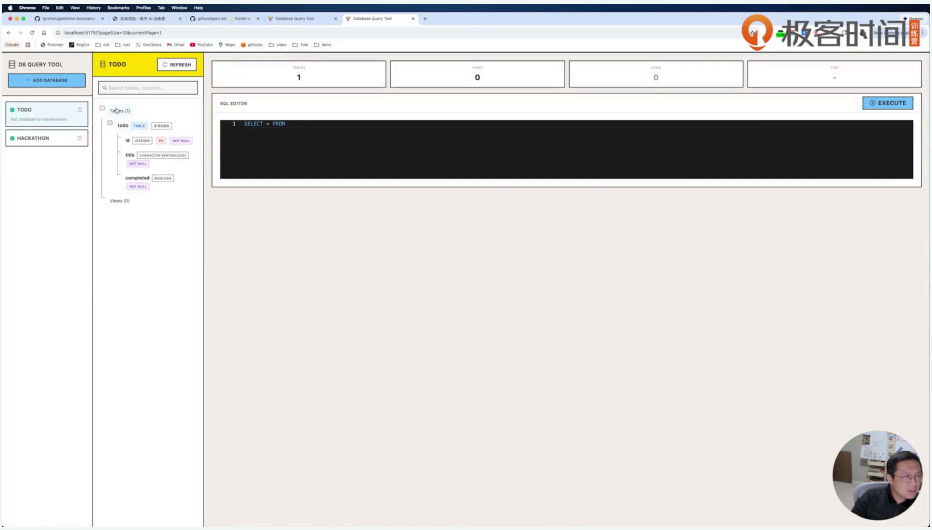
> **做完的标准**： 用户可以不用猜测当前上下文完成一次查询；页面结果与 API 响应一致，失败状态可定位到具体环节。

- [ ] 核对加载、空结果、成功、失败四种界面状态。
- [ ] 核对页面列名与 API 返回，记录失败所在环节。

按真实组件找入口

现码主界面在 `frontend/src/pages/Home.tsx`。`DatabaseSidebar` 管连接，`MetadataTree` 展示结构，`SqlEditor` 管 SQL，`NaturalLanguageInput` 输入中文或英文需求。主界面按当前代码包含 `REFRESH`、`GENERATE SQL`、执行和导出按钮；课程画面与现码样式可能不同，以图注标明历史状态。[O6]

点表时当前代码生成 `SELECT * FROM ${schemaName}.${name} LIMIT 100`，没有按方言正确引用所有特殊标识符。表名有空格、保留字或大小写要求时，先在编辑器改正；不要把点击表产生的示例 SQL 当作任何名字都能执行的模板。



图注：真实课程视频帧，w2第5讲01:01:40；编辑器中的SQL尚未完整填好，结果区不证明当前草稿已执行成功；2026-10-29提取/核看，保留录课水印，不是本次实机结果。

保留两条跨项目原输入

`.chats/w1.md` 最后两节谈“数据库列表侧栏”和 schema UI，虽然放在 w1 档案里，内容属于数据库工具。完整保留为 W1-P07、W1-P08，未混入 Project Alpha 的功能要求。前者要求能看到所有数据库，后者要求 schema 列表清晰、schema 占满可用高度、结果区占满空间。这些是迭代目标，仍需对照当前页面行为验收。

助手模板 `T06-ui-review.md`：

> 场景：〔添加连接/切库/展开schema/查询/错误/导出〕
> 当前界面与截图：〔附图与时间〕
> 具体问题：〔可复现描述〕
> 期望布局/交互：〔元素、空间、状态〕
> 保持API与数据逻辑，先列相关组件，再做最小修改。
> 请用可复现步骤检查成功、空、加载、失败四态，附实际运行证据。

步骤 7 用元数据生成 SQL，并保留人工执行门槛

模型需要真实结构才能写出有依据的SQL。生成之后还要检查查询语义；一条能运行的SQL也可能回答了不同问题。

目标： 把自然语言需求转换成有结构依据的 SQL，不直接盲执行模型输出。

1. 确认演示库元数据已成功获取且未过期，检查结构中包含本次需求用到的表和列。[O7]

2. 在后端本机私有配置中提供实际可用 key，重新启动使配置生效；先查看模型调用代码与计费条件，再执行一次低量练习。[O7]
3. 输入具体查询条件，让模型生成 SQL；核对字段、过滤、连接、日期范围、方言和 LIMIT，发现不符先修改或重新生成。[O7]
4. 在手写 SQL 编辑器中检查后主动执行，比较数据结果与自然语言原意；只生成成功还不算完成查询。[O7]
5. 把需求、元数据摘要、生成 SQL、你的修订与实际结果写入模板，记录模型错误而不是只保留成功例子。[O7]

> **做完的标准**： 有一条"需求→元数据→生成SQL→人工核对→执行结果"的闭环，且未把 key 暴露给前端或记录文件。

- [ ] 保存需求、生成 SQL、人工审核意见及执行结果。
- [ ] 标明模型未调用、生成失败或语义错误的停点。

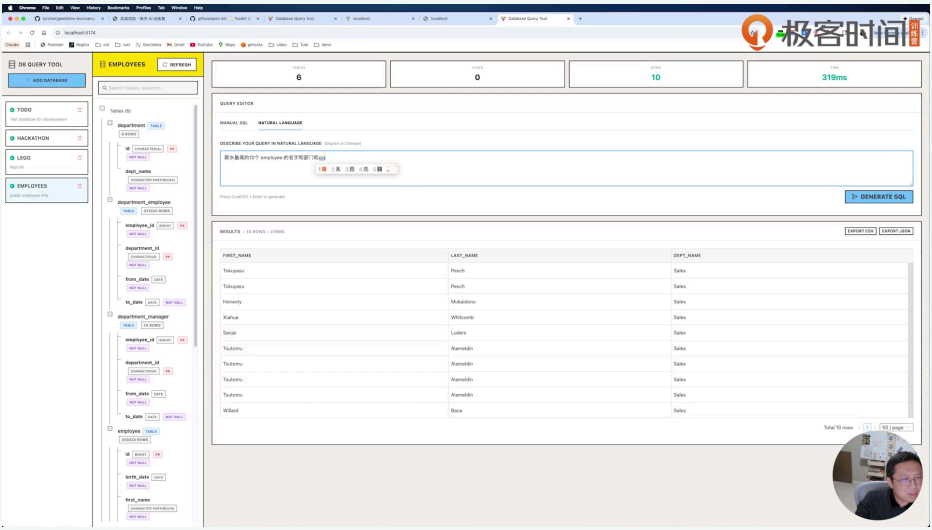
运行时提示词怎样组成

系统提示词由 `NaturalLanguageToSQLService._build_prompt()` 生成，包含数据库类型、表/视图与列描述、只输出 SELECT、无 LIMIT 时加 1000、PostgreSQL/MySQL 的语法要求；用户消息就是 `user_prompt`。源码固定模型 `gpt-4o-mini`、temperature 0.1、max_tokens 500，属于课程历史实现，不代表当前最适合或永远可用。完整构造代码见运行时原文。[O7]

现码对模型返回做围栏剥离，但自然语言接口不会自动执行 SQL；前端把生成结果放入编辑器并切换到手写页签，提示用户编辑后执行。后端返回的 `explanation` 是 `Generated SQL from: ...` 的文本拼接，不是模型经过核查的推理说明。

```
curl --fail-with-body -X POST
http://127.0.0.1:8000/api/v1/dbs/demo_pg/query/nat
ural \
  -H 'Content-Type: application/json' \
  -d '{"prompt":"查询演示任务中尚未完成的记录，返回编号和
标题，最多10条"}'
```

这条请求会使用真实模型 key，不能在只做离线复核时直接运行。本次制作未调用真实模型，因此外部服务可用性与生成效果留给读者按此步骤实测。



图注：真实课程视频帧，w2第5讲01:40:05；讲者还在修改自然语言条件，下方旧结果有重复人员；不能作本次新输入成功或语义正确证明；2026-10-29提取/核看，保留录课水印，不是本次实机结果。

原提示词与本步模板

功能要求来自 W2-P02，MySQL 条件来自 W2-P05。下面是助手为查询需求提供的填写格式，保存为 `T07-natural-language-query.md`：

```
目标库/方言：〔PostgreSQL或MySQL〕
已存在的表与列：〔从元数据复制，不填密码〕
我要回答的问题：〔业务问题〕
筛选条件：〔状态/日期/其他〕
结果列：〔列名〕；排序：〔字段和顺序〕；最多：〔条数〕
不确定之处：〔需要先澄清的词义〕
生成后先核对SQL，不直接执行。
```

如果元数据返回404，先刷新；401/额度/模型不可用等属于实际外部服务问题，保存脱敏错误，先继续完成不依赖模型的手写查询路径。别用假 SQL 响应伪装完成真实生成验收。

步骤 8 增加 MySQL，并检视扩展架构

第二种数据库会暴露原先只为PostgreSQL写的隐含假设。先让MySQL工作，再用两种库回归约束架构修改。

目标： 以第二种数据库检验抽象是否足够，同时保持原 PostgreSQL 路径可用。

1. 阅读原 W2-P05、W2-P06 和 w3 扩展课，确认本次新增的是连接、元数据、执行与自然语言方言四个方面。[O8]
2. 准备独立 MySQL 演示库，用脱敏连接串添加；按与 PostgreSQL 同一验收表跑连接、结构、手写查询和生成 SQL。[O8]

3. 对照旧 connection_factory 分支实现与当前 Adapter/Registry/Facade 结构，沿实际路由确认哪些新旧代码仍被调用。[O8]
4. 用 W3-P01 建评审命令，再用 W3-P02 的目标核架构；对报告建议逐项找代码证据，不直接接受分数和“零风险”等措辞。[O8]
5. 修复时先做一项有依据的缺口并回归两类库；把没测试的功能单列，不把一次 MySQL 成功推为全部数据库都可扩展。[O8]

> **做完的标准**： 两类数据库有同口径验收记录；你能画出实际查询调用链，并指出新增数据库还要修改的类型、校验、提示词和UI位置。

- [ ] 分别记录 PostgreSQL 与 MySQL 的同口径回归。
- [ ] 检查前端 URL 校验、方言提示词和无面试语义。

两种方言的差别

部分	PostgreSQL	MySQL
驱动	asyncpg	aiomysql
URL	postgresql://...	mysql://...
sqlglot dialect	postgres	mysql
常见标识符引用	双引号	反引号
系统结构信息	pg_catalog/information_schema	information_schema
示例	`public.demo_items`	`demo_db.demo_items`，以实际元数据为准

原提示词中的 `todo_db`、`interview_db` 和 `mysql://root@localhost:3306/interview_db` 是讲者本地示例，不代表你的电脑也有，不要原样当作可用环境。源码自带创建和种子 SQL 只应在你新建的演示环境里使用，先审查其 DROP/CREATE/INSERT 行为。

MySQL课的实际扩展与回归顺序

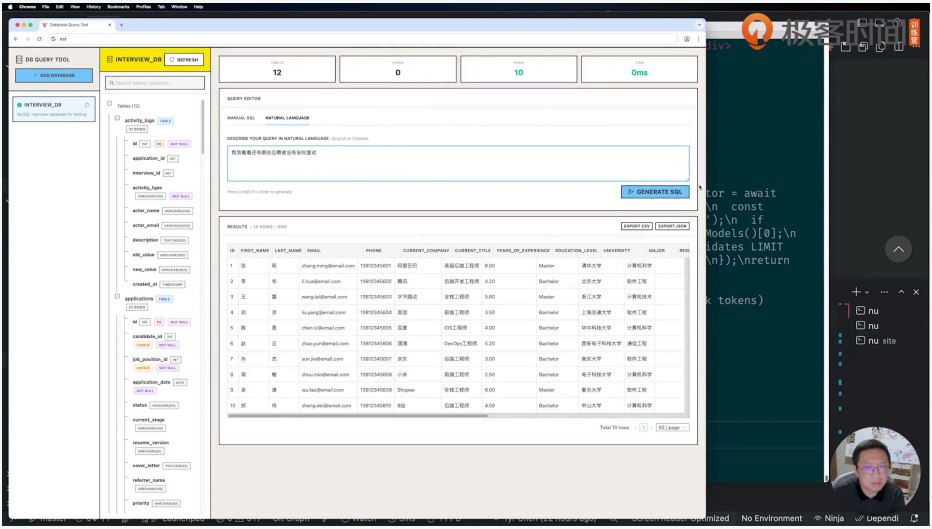
1. 先用讲者本地todo_db简单表验证MySQL连接、metadata与执行，再使用interview_db的多表数据测试自然语言联查；两个库用途不同，不把“todo成功”当复杂关系已验证。 [O10]
2. 按W2-P05给现有PostgreSQL实现增加MySQL。课中SpecKit生成任务的位置不合意，讲者移动文件并告知新路径；新工具若仍读旧feature目录，先更正任务定位再implement，不在错目录继续。 [O10]

3. 用W2-P06把MySQL请求加入正确fixtures/test.rest，后端通过后再测试界面。课中00:15:35发现前端只允许PostgreSQL URL，所以后端驱动支持还不等于UI支持；URL/schema校验、方言和模型提示词都要核。 [O10]
4. 用虚构招聘数据做一个明确问题，再审查生成SQL，主动执行。完成MySQL后，按课中00:19:21–00:20:16回到PostgreSQL重测，防止修改MySQL分支破坏旧路径。 [O10]
5. 功能回路成立后再看架构。课中00:21:11看到if/else到处加，才要求接口抽象；不要只给模式名字而没有减少哪些真实修改点的证据。[O10、O8]

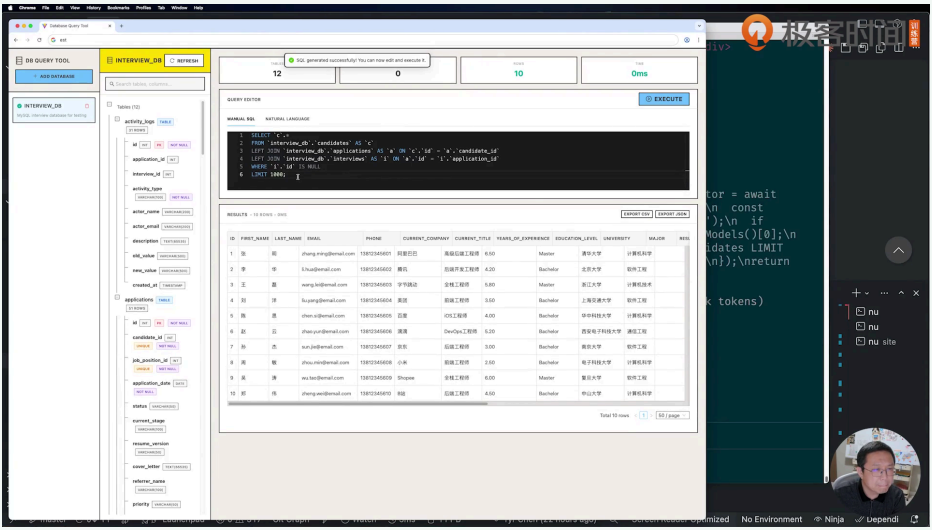
屏幕原输入 DB-V09｜MySQL自然语言测试

w3第4讲00:19:00，输入完整；下方表格还是先前结果，不视为此问题结果。状态：完整可见，不是从ASR拼出的逐字全文。[O11]

> 帮我看看还有哪些应聘者没有安排面试



图注：真实课程视频帧，w3第4讲00:19:00，输入完整；下方表格还是先前结果，不视为此问题结果；2026-10-29提取/核看，保留录课水印，不是本次实机结果。



图注：真实课程视频帧，w3第4讲00:19:05；SQL生成成功提示可见，结果表仍为旧10行；此刻不能证明这条新SQL已经执行；2026-10-29提取/核看，保留录课水印，不是本次实机结果。

这条“没有安排面试”SQL为什么还要审语义

屏幕生成的是从candidates左联applications，再左联interviews，用`i.id IS NULL`筛选。它可能表示“存在一份没有面试的申请”，未必等于“这个人从未有任何申请安排过面试”。一人多申请还可能重复。先问清按人还是申请、是否只看当前状态；再考虑NOT EXISTS、去重或指定申请范围。此处是助手对画面SQL的语义分析，不宣称课程已经修正或本次实库验证通过。

实际架构与文档的距离

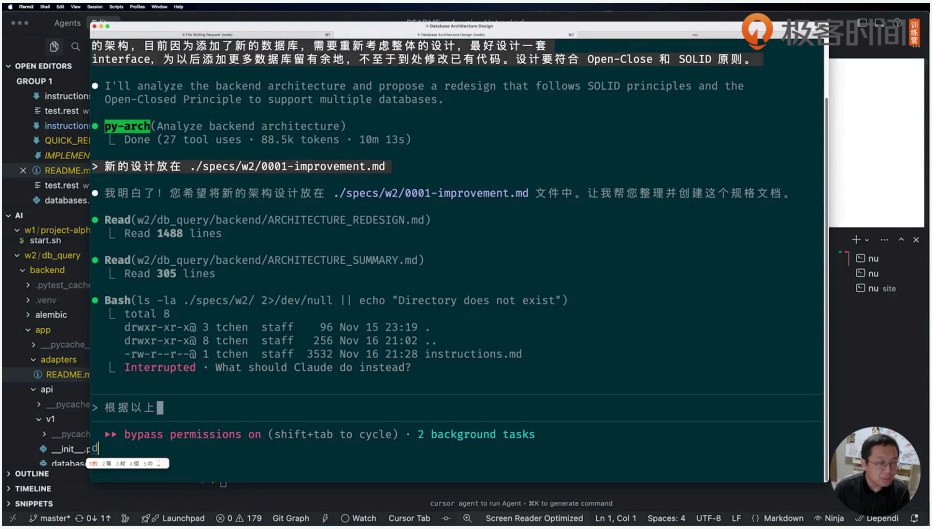
课程重构文档组合 Adapter、Factory、Registry、Facade：接口处理 HTTP，DatabaseService 协调校验与调用，Registry 选适配器，适配器负责驱动/元数据/查询。元数据路由仍调用旧 `fetch_metadata→connection_factory`，查询路由则走新适配器体系，重构并未消除所有旧分支。当前 `DatabaseType` 仍只有 PostgreSQL 与 MySQL，新增 Oracle 不能只复制文档里"注册一行"就完成；还要处理类型、URL校验、SQL方言、模型提示词和前端选项。

Registry 的缓存键是数据库类型加连接名称，不包含 URL；更新同名连接后要检查旧适配器/连接池是否被正确释放，否则可能仍指向旧连接。原架构报告给出的代码量、重复率、工期和健康分数是当时报告自述，本次没有独立测量，不能当作验证结果。[O8]

屏幕原输入 DB-V08｜指定架构文稿落点

w3第4讲00:30:10仍可见已发送消息，文件历史路径保留。状态：完整可见，不是从ASR拼出的逐字全文。[O11]

```
新的设计放在 ./specs/w2/0001-improvement.md
```



图注：真实课程视频帧，w3第4讲00:30:10仍可见已发送消息，文件历史路径保留；2026-10-29提取/核看，保留录课水印，不是本次实机结果。

原提示词与本步模板

本阶段四条原输入全部在附录。助手模板 `T08-mysql-and-architecture.md`：

目标：给〔当前项目〕增加MySQL支持，同时保持PostgreSQL行为。
先沿真实路由列出连接、元数据、SQL验证、执行、NL提示词、前端类型的调用链。
对比旧分支方案与适配器方案，指出必须改的文件，不为模式名称增加空抽象。
验收矩阵：两类库×连接/刷新/手写SQL/自然语言/错误/历史。
每条只登记实际运行证据；外部DB或模型未测时明确写未测。

步骤 9 测试、评审与打包复现说明

把实际结果与模型宣告分开，失败日志同样是交付的一部分。下一位接手者应该能定位停点并继续。

目标： 交付一份别人能判断当前完成度的项目，而非一串 AI 的成功宣告。

1. 在隔离的配置目录运行后端单元测试，前端做类型/构建检查和一次性测试；保留完整输出及退出码。[O9]
2. 用演示库按接口顺序测试，再用界面复核同一场景；区分模拟测试、真实数据库集成测试和真实模型调用。[O9]
3. 逐条处理失败，不因测试文件存在就说已覆盖；重构后先检查 mock 的符号名与函数签名是否仍匹配。[O9]
4. 按原提示词 W2-P03 做 curl 和页面回归，补上空态/失败态与数据库切换，留下截图。[O9]

5. 打包说明环境、启动目录、配置方式、数据初始化、命令、已通过项和未完成项；填写最终验收模板后再交付。[O1、O8、O9]

> **做完的标准**： 每个“通过”都能对应日志、响应或截图；未做的真实调用写未做，后续接手者能复现当前状态。

- [ ] 每条通过结论指向日志、响应或真实截图。
- [ ] 在 T09 列出所有未测项、已知失败与复现命令。

推荐的检查次序

后端目录中环境隔离好再执行：

```
uv run pytest tests/unit -q
uv run mypy app
uv run ruff check app
```

前端目录：

```
npm run build
npm test -- --run
```

`npm test` 的脚本是 vitest，直接运行可能进入监听；一次性检查加 `--run`。`npm run lint` 当前脚本与 ESLint 配置/版本是否适配，需按实际报错检查。上面是验证步骤，不表示本次制作已经全部跑通。

analyze、修复与作业要分开记录

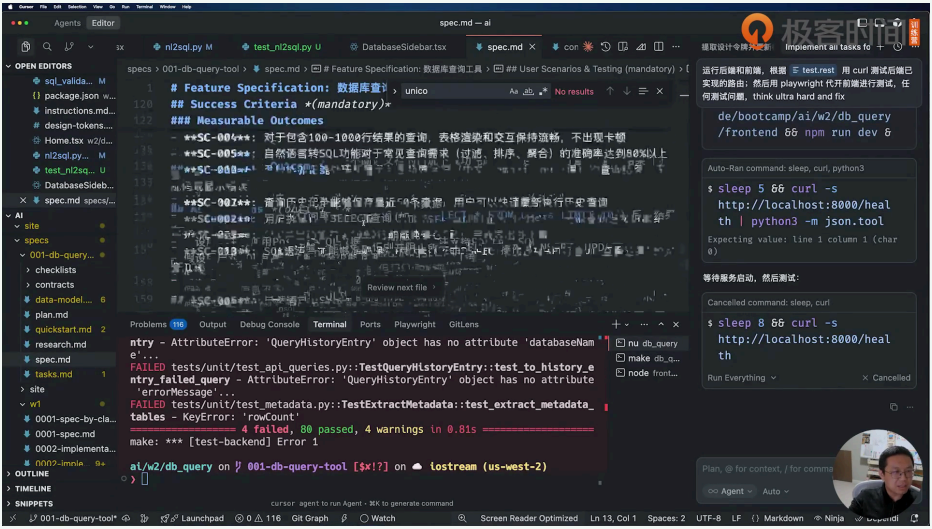
第5讲01:41:00之后使用analyze检查已有代码，01:44:00要求分析必须存成文档，随后处理连接池关闭、UTC时区、导出与测试。操作时先保存报告，按“问题→触发条件→受影响文件→最小修复→回归”登记，不让修复意见只留在聊天里。报告写了机会，不等于每项已实现。[O10]

课程还在01:46:00提醒Python中批量文本替换需要测试覆盖；不要为了快速消灭报错，用大面积sed替换接口却不核调用。单元测试失败先分成产品行为变了、测试mock过期、fixture不完整或真实回归，核产品契约后再改。对当前20条失败，本手册只核到实际日志与若干接口漂移线索，未宣称全部失败根因已查清。[O9]

CSV/JSON导出在早期spec里被模型加进需求，而第5讲01:55:00仍被讲者作为课后完善作业。**历史录课阶段**与**当前源码快照**不同：当前前端已有导出入口，仍须核导出的是哪次查询/格式/NULL/空结果；不能把后来按钮存在倒写成当堂已全验收。w3架构课最后只是下达重构所有phase与测试的要求，没有展示完整收尾。[O10]

本次制作实际检查结果

检查范围	实际结果	证据与限制
原后端安装	缺README，打包失败	practice-install.log
副本最小补README后运行依赖	uv sync --frozen --offline成功，安装36包	practice/backend-runtime-install.log；不等于接口已测
7例SQL解析	已保存每例validate与转换结果	practice/sql-boundary-results.json；未执行数据库
后端单元测试	**64 passed / 20 failed / 4 warnings**	practice/backend-unit-test.log；不能写全通过
前端npm ci	package/lock不一致失败	practice/frontend-install.log
前端修正安装尝试	网络过程被中止，未完成构建	practice/frontend-install-repair.log
真实PostgreSQL/MySQL与模型	本次未验收	课程画面只证明讲者当时做过
新手从头照做	not_tested	不用本文内容核对代替



图注：真实课程视频帧，w2第5讲02:00:00；终端显示4 failed、80 passed，是讲者当时尚有失败的测试画面，并非数据库界面成功证据；2026-10-29提取/核看，保留录课水印，不是本次实机结果。

可套用模板 T15：评审到修复（助手补写）

请对照〔spec/plan/tasks〕分析〔真实代码目录〕，把完整报告保存〔路径〕。
每条写：具体文件/行、触发输入、违反的契约、可复现方法、建议最小改动。
先按产品缺陷/过期测试/环境/未验证区分，不靠改断言让测试变绿。
我这轮只修〔一个问题〕；请记录改前、改后与相同场景重测输出。
旧PostgreSQL与新MySQL都要回归；真实DB/模型没跑就写未测。

源码测试存在历史接口漂移

例如 `test_api_queries.py` 仍 patch `app.api.v1.queries.execute_query`，而实际路由导入的是 `execute_query_with_service`；其断言和 NL 调用参数也有重构前形态。这类失败首先是测试与调用路径不一致，应先确认产品契约，再更新模拟目标与断言；不能为了让旧测试绿灯把已确认的新接口倒退。[O9]

`T09-final-acceptance.md` 给出"需求、场景、步骤、预期、实际、证据、状态、阻塞和下一动作"列。`T10-review-request.md` 要求评审者报告文件位置、问题、依据、复现和建议优先级。原作者要求"think ultra hard"不能替代这些具体证据。

六个课堂复现案例：怎样从现象走到下一步

这些案例记录课堂观察和本次材料核对；它们不代表本机已经连通数据库或调用模型。每个案例都可复制到 T01/T15，填入自己的命令、响应与下一步。

案例 1：uv 安装在业务代码运行前失败

起点是 L05 的 `backend uv sync 出错`。先读错误末尾，再核对 `pyproject.toml` 的 readme 路径和实际文件；本次副本确认缺 `backend/README.md`，加标注的占位文件后离线 runtime 安装通过。这里修的是打包元数据，仍需另测迁移、健康检查和查询。记录模板：错误原文→声明路径→实际文件→最小修补→重跑日志。[O1] [O9]

案例 2：生成的测试 REST 文件来自别的项目

L05 生成测试入口时屏幕出现与本项目不符的标签，老师指出拿错内容。先核对 base URL、路由、请求体与项目名，再拿真实 OpenAPI/代码确认，最后逐条发请求。文件成功生成不等于接口验证完成；不要复制陌生端点作为验收证据。记录模板：可疑字段→正确路由→修订差异→真实响应。[O10]

案例 3：Cursor 额度耗尽后换工具

L05 约 18 分钟后从 Cursor 切到 Claude Code。课堂延续同一份规格和任务，没有重新发散需求。交接时给出工作目录、已完成 phase、未提交差异、测试 REST 与下一条验收动作；新工具先读文件，再继续同一任务。本册保留课程命令的历史形式，实际可用模式以本机安装为准。记录模板：工具→停点→文件清单→当前失败→下一动作。[O10]

案例 4：升级依赖后白屏

L05 既有升级所有依赖的输入，也有前端兼容性反复修正。先保留锁文件和报错，再分别查终端、浏览器 Console、Network 与根组件挂载，才决定是否固定某个版本。React 18.3.1 是课堂的排错尝试，不能作为无条件降级答案。本次 `npm ci` 仍遇锁文件漂移，修复安装因网络中断，没有构建结论。记录模板：升级差异→首个错误→证据→一次最小变更→重跑。[O9] [O10]

案例 5：SQL 执行成功，却重复返回员工

L05 从 todo 到 Lego、员工薪资问题逐步增加查询难度；自然语言截图中输入仍在修改，下方旧结果有重复人员。先定义一行应代表什么，再查 JOIN 基数、分组及窗口排名；不要用“返回了行”代替“回答了问题”。用 T17 写出预期人员集合和反例，重跑后逐行核对。[O10]

案例 6：MySQL 后端已改，前端仍拒绝 URL

W3 L04 出现后端增加 MySQL 后前端校验还只接受 PostgreSQL 的遗漏。测试应从 URL 输入到连接、metadata、手写 SQL、自然语言再回归 PostgreSQL，不能只运行后端单测。“未安排面试”还需区分从未有面试、没有未来面试和没有待办面试；同一个 LEFT JOIN 不自动满足三种定义。架构重构则先保存设计文稿，再逐阶段实现，课堂并未展示全套重构完工。[O8] [O10]

可填写模板与材料包

本手册对应的 `templates/` 文件夹提供原提示词、Spec Kit 原始模板/命令、课程生成文档和助手新增工作表。占位项必须填成真实项目条件后再使用。原文件中的历史路径、语法或疑问保留，使用前按本书说明适配。

模板	用途	性质
T01-environment-and-debug.md	安装、配置、根因与修补日志	助手新增
T02-cursor-task.md	限定一次AI改动范围	助手新增
T03-spec-to-tasks.md	需求到验收矩阵	助手新增
T04-connection-metadata.md	连接/结构核对	助手新增
T05-sql-boundaries.md	SQL边界纯解析测试表	助手新增
T06-ui-review.md	界面四态与操作证据	助手新增
T07-natural-language-query.md	生成、审查、执行记录	助手新增
T08-mysql-and-architecture.md	第二数据库与架构回归	助手新增
T09-final-acceptance.md	最终验收	助手新增
T10-review-request.md	有证据的代码评审	助手新增
T11-rule-and-command-lab.md	规则与重复命令实验	助手新增
T12-design-rules-lab.md	风格规则与设计纠偏	助手新增
T13-memory-and-spec-lab.md	Memory Bank 与 Spec Kit 实验	助手新增
T14-phase-implementation.md	三阶段实现与工具交接	助手新增
T15-analyze-and-repair.md	评审、修复与回归	助手新增

模板	用途	性质
T16-mysql-real-regression.md	双数据库真实回归	助手新增
T17-query-semantics.md	结果语义与重复行核对	助手新增
T18-screen-input-card.md	完整/部分屏幕输入档案	助手新增
W-*P*-original.md	每条作者原输入	原文快照
runtime-nl2sql-original.py	项目实际运行时消息构造	原代码
cursor-original-commands/、cursor-original-rules/	工作流命令与项目规则	原工程文件
spec-kit-original-templates/	规格、任务等母版	原工程文件
course-spec-kit-output/	本项目已生成的规格材料	课程产物，非空白母版

扩展边界与排错索引

现象	先查什么	下一动作
安装前就报README不存在	backend/pyproject声明与实际文件	在副本最小修补包元数据
启动报key缺失	Settings导入与.env位置	先配置，离线假key不验收生成
8000正常但5173无页面	前端终端是否启动	不依赖顺序make dev同时启动
添加连接失败	URL、端口、权限、库是否存在	用本机DB客户端验证演示库
元数据为空/失败	缓存、schema权限、刷新响应	单独检查GET/refresh
NL提示缺metadata	缓存不存在或过期	先GET结构或强制刷新
SQL显示正确但执行失败	方言、schema、引号、列名	手工检查生成SQL再执行
测试patch找不到符号	真实router导入和测试mock	更新到实际调用路径
同名连接换URL仍像旧库	Registry与池缓存生命周期	检查关闭/替换旧实例
导出与画面不一致	查询结果是否旧状态	固定本次响应后再导出

在联网部署前，至少重新处理认证、网络绑定、连接串存储、目标地址限制、只读数据库角色、超时、查询资源限制、模型数据传输与审计。这里列的是从本项目原型边界推出的工程待办，不声称课程已经实现；本书交付目标仍是本地演示工具。

作者原提示词全集

以下十条来自本次项目相关聊天档案，按文件顺序保留全文，未将助手模板冒充原话。每条前注明原文件与行范围；档案顺序不必然等于视频操作顺序。`@agent-py-arch`、`think ultra hard` 等历史输入按原样保存，并不代表本手册制作过程调用了相应代理。

W2-P01 constitution

原文件 `_raw/chats/w2.md` 第 4–12 行；段落 SHA-256：`a24721b4b070d102c204a16c3252ccbd1fc3ecbdcb1e407f909da6aac23add15`。

```
这是针对 ./w2/db_query 项目的:

- 后端使用 Ergonomic Python 风格来编写代码，前端使用 typescript
- 前后端都要有严格的类型标注
- 使用 pydantic 来定义数据模型
- 所有后端生成的 JSON 数据，使用 camelCase 格式。
- 不需要 authentication，任何用户都可以使用。
```

W2-P02 基本思路

原文件 `_raw/chats/w2.md` 第 14–59 行；段落 SHA-256：`ae34ac17ffccbbe1762529a421278251e9e0e92bf599a7411f223bf57be5c324`。

这是一个数据库查询工具，用户可以添加一个 db url，系统会连接到数据库，获取数据库的 metadta，然后将数据库中的 table 和 view 的信息展示出来，然后用户可以自己输入 sql 查询，也可以通过自然语言来生成 sql 查询。

基本想法：

- 数据库连接字符串和数据库的 metadata 都会存储到 sqlite 数据库中。我们可以根据 postgres 的功能来查询系统中的表和视图的信息，然后用 LLM 来将这些信息转换成 json 格式，然后存储到 sqlite 数据库中。这个信息以后可以复用。
- 当用户使用 LLM 来生成 sql 查询时，我们可以把系统中的表和视图的信息作为 context 传递给 LLM，然后 LLM 会根据这些信息来生成 sql 查询。
- 任何输入的 sql 语句，都需要经过 sqlparser 解析，确保语法正确，并且仅包含 select 语句。如果语法不正确，需要给出错误信息。
  - 如果查询不包含 limit 子句，则默认添加 limit 1000 子句。
- 输出格式是 json，前端将其组织成表格，并显示出来。

后端使用 Python (uv) / FastAPI / sqlglot / openai sdk 来实现。
前端使用 React / refine 5 / tailwind / ant design 来实现。sql editor 使用 monaco editor 来实现。

OpenAI API key 在环境变量 OPENAI_API_KEY 中。数据库连接和 metadata 存储在 sqlite 数据库中，放在

~/.db_query/db_query.db 中。

后端 API 需要支持 cors，允许所有 origin。大致 API 如下：

```bash
# 获取所有已存储的数据库
GET /api/v1/dbs
# 添加一个数据库
PUT /api/v1/dbs/{name}

{
  "url":
"postgres://postgres:postgres@localhost:5432/postg
res"
}

# 获取一个数据库的 metadata
GET /api/v1/dbs/{name}

# 查询某个数据库的信息
POST /api/v1/dbs/{name}/query

{
  "sql": "SELECT * FROM users"
}

# 根据自然语言生成 sql
POST /api/v1/dbs/{name}/query/natural
```

````
{
  "prompt": "查询用户表的所有信息"
}
```
````

W2-P03 测试

原文件 `_raw/chats/w2.md` 第 61–63 行；段落 SHA-256：`ba3303eded458946a3c5f277a0bfbf46d074f9649212c4d95f28bbb1e2b63515`。

```
仔细阅读 ./w2/db_query 下面的代码，然后运行后端和前端，根据@w2/db_query/fixtures/test.rest 用 curl 测试后端已实现的路由；然后用 playwright 开前端进行测试，任何测试问题，think ultra hard and fix
```

W2-P04 db migration & unit test

原文件 `_raw/chats/w2.md` 第 65–67 行；段落 SHA-256：`9e9a1653b68925fb0fd3d61c0db910f71e44b5baf090e794b26d723793c7376c`。

```
`make setup` 会出错，修复它；确保前后端 unit test 都通过。
```
```

W2-P05 添加 MySQL db 支持

原文件 `_raw/chats/w2.md` 第 69–71 行；段落 SHA-256：`eaabba56ef97bcda236a794cf28e5bb7101fb781669762eaa3548d251fc61bad`。

> 参考 ./w2/db_query/backend 中的 PostgreSQL 实现，实现 MySQL 的 metadata 提取和查询支持，同时自然语言生成 sql 也支持 MySQL。目前我本地有一个 todo_db 数据库，使用 `mysql -u root todo_db -e "SELECT * FROM todos;"` 可以查询到数据。

W2-P06 测试 MySQL db 支持

原文件 `_raw/chats/w2.md` 第 73–78 行；段落 SHA-256：`0a209abb6bb81711c03acfb2d6061d3ea4468eea2fb838dda482ca8bdcfffe28`。

```
目前 mysql 已经得到支持，在 ./w2/db_query/fixtures/test.rest 中添加 MySQL db 支持的测试用例，然后运行测试。如果后端测试 ok，那么打开后端和前端，使用 playwright 测试前端，确保 MySQL db 的基本功能：

- 添加 新的数据库 interview_db（url 为 mysql://root@localhost:3306/interview_db）
- 生成 sql，查询 interview_db，并显示结果
- 自然语言生成 MySQL sql，查询 interview_db，并显示结果
```

W3-P01 code review command

原文件 `_raw/chats/w3.md` 第 4–11 行；段落 SHA-256：`df79df1d3f3e9dd5d5eb19f1ff4a1977f9ec945e3782dd53610bb4eaadc72c00`。

帮我参照 @.claude/commands/speckit.specify.md 的结构，think ultra hard，构建一个对 Python 和 Typescript 代码进行深度代码审查的命令，放在 @.claude/commands/ 下。主要考虑几个方面：

- 架构和设计：是否考虑 python 和 typescript 的架构和设计最佳实践？是否有清晰的接口设计？是否考虑一定程度的可扩展性
- KISS 原则
- 代码质量：DRY, YAGNI, SOLID, etc. 函数原则上不超过 150 行，参数原则上不超过 7 个。
- 使用 builder 模式

W3-P02 review 代码

原文件 `_raw/chats/w3.md` 第 13–16 行；段落 SHA-256：`cdc2e870ec2849055670f1526e87d886caeef1833ecbfed21421fb396ffc8920`。

@agent-py-arch 帮我仔细查看 ./w2/db_query/backend 的架构，目前因为添加了新的数据库，需要重新考虑整体的设计，最好设计一套 interface，为以后添加更多数据库留有余地，不至于到处修改已有代码。设计要符合 Open-Close 和 SOLID 原则。

W1-P07 侧边栏

原文件 `_raw/chats/w1.md` 第 35–37 行；段落 SHA-256：`3ff3f49062da029f713cb654ddddc9ece71709a2771d1888630aaf5a9abaa0f3`。

> 侧边栏目前用途不大，可以放所有已有的数据库，并且把添加数据库，删除已有的数据库的功能放在侧边栏。侧边栏也要使用新的 token 风格。然后主页直接显示第一个数据库的 metadata 信息和查询界面，这样用户可以减少一次点击进入到 database display 页面。一个页面囊括所有功能。

W1-P08 schema tree view

原文件 `_raw/chats/w1.md` 第 39–40 行；段落 SHA-256：`cd764771b047d0f49562d8fb6ed94d837a1c43c0d80a726546d5d48bcababfac`。

> [Image #2] schema 边栏应该直接铺满除了db name 外整个高度，db name + refresh button 是一个 top bar，使用偏黄色的 token style color；schema 内部的 treeview 应该铺满整个 schema；[Image #3] 这些 metrics 和main sql editor 放在一个容器，他们可以更小一些，不需要占这么多空间。

课堂屏幕输入与未展示范围

聊天文件十条完整原文均保留在上一章；它们没有涵盖全部屏幕输入。以下表记录这次实际核到的新增输入，完整文字已在对应动作处呈现；输入仍在编辑的只记录可见部分，不用逐字稿补出未显示内容。[O11]

ID	用途	画面定位	状态
DB-V01	生成MotherDuck规则	w2第2讲 00:32:35	full；第2讲 00:32:35，尚未附加后续参考要素，不把后续内容并进这句
DB-V02	继续提取设计token	w2第3讲 00:07:03	full；第3讲 00:07:03可见上次消息回顾；链接/附件标签转为文件名，软折行合并
DB-V03	Memory Bank初始化	w2第3讲 00:09:50	full；第3讲 00:09:50，历史工具实验，不要求数据库项目安装它
DB-V04	限定任务阶段	w2第4讲 00:39:30	full；第4讲 00:39:30；输入完整，代理任务说明与总数是另一部分
DB-V05	uv安装失败反馈	w2第5讲 00:03:10	full；第5讲 00:03:10，屏幕同时显示缺README错误与后续重跑，不能把代理成功宣告当本次安装结果
DB-V06	升级依赖	w2第5讲 00:04:30	full；第5讲 00:04:30；这是课程输入，复现现成代码不照此盲目升级

ID	用途	画面定位	状态
DB-V07	实现第二阶段	w2第5讲 00:10:50	full；第5讲 00:10:50，保留大小写与phase任务范围
DB-V08	指定架构文稿落点	w3第4讲 00:30:10	full；w3第4讲 00:30:10仍可见已发送消息，文件历史路径保留
DB-V09	MySQL自然语言测试	w3第4讲 00:19:00	full；w3第4讲 00:19:00，输入完整；下方表格还是先前结果，不视为此问题结果
DB-Q01	Phase1输入可见部分	w2第5讲 00:00:55	partial；底部输入被画面/讲者遮挡，不能从ASR补成完整句
DB-Q02	Phase3输入准备	w2第5讲 01:03:00	partial；第5讲 01:03:00仅可见命令/补全，后续阶段文字未在此帧完整显示
DB-Q03	MySQL测试库输入草稿	w3第4讲 00:10:00	partial；末尾真实seed等字尚在输入，所见即止，不造完整原话

ID	用途	画面定位	状态
DB-Q04	架构实施输入草稿	w3第4讲 00:30:40	partial；指令仍在输入，后续implementation guide/所有phase等按口述转述而非原prompt

还未逐条认证的范围包括：第1讲Rust输入全句/图片优化命令生成输入，第2讲长篇设计参考附件、样式纠偏多轮，第3讲Memory Bank计划输入，第4讲plan长输入与constitution多段，第5讲Makefile/REST生成、Phase1/3完整输入、analyze完整原命令/报告落点、模型参数修改与截图指令，w3移动tasks、复杂seed和最终重构实施完整句。对应动作在正文按口述或源码转述，**不标成完整作者原prompt**。需要它们时回到对应视频连续画面核全文和续图。

下面是可见草稿的原样转写，分别保留边界，不作为可直接执行的完整任务：

DB-Q01｜Phase1输入可见部分（部分）

```
/speckit.implement 完成 phase
```

底部输入被画面/讲者遮挡，不能从ASR补成完整句。图路径：`images/w2-l05-0055.png`。

DB-Q02｜Phase3输入准备（部分）

```
/speckit.implement
```

第5讲01:03:00仅可见命令/补全，后续阶段文字未在此帧完整显示。图路径：`images/w2-l05-3780.png`。

DB-Q03｜MySQL测试库输入草稿（部分）

```
构建一个相对复杂的 mysql 数据库 interview_db，记录公司招聘面试安排面试结果等相关的信息，并添加足够丰富
```

末尾真实seed等字尚在输入，所见即止，不造完整原话。图路径：`images/w3-l04-0600.png`。

DB-Q04｜架构实施输入草稿（部分）

```
根据以上的重构设计
@w2/db_query/backend/ARCHITECTURE_REDESIGN.md
```

指令仍在输入，后续implementation guide/所有phase等按口述转述而非原prompt。图路径：`images/w3-l04-1840.png`。

运行时提示词与消息构造原代码

下面完整保留课程 `app/services/nl2sql.py`，包含系统提示词两种方言分支、用户消息、模型参数、返回清理与错误处理。它是项目程序实际使用的提示词构造，不是讲者发给编程工具的一条聊天输入。`schema_text` 运行时由元数据填入，不能把变量名当成最终完整消息。[O7]

```
"""Natural Language to SQL conversion service
using OpenAI."""

from openai import AsyncOpenAI
from app.config import settings
from app.models.database import DatabaseType
import logging

logger = logging.getLogger(__name__)


class NaturalLanguageToSQLService:
    """Service for converting natural language
queries to SQL using OpenAI."""

    def __init__(self):
        """Initialize OpenAI client."""
        self.client =
AsyncOpenAI(api_key=settings.openai_api_key)
        self.model = "gpt-4o-mini"  # Cost-
effective model for SQL generation

    def _build_prompt(
        self, user_prompt: str, metadata: dict,
db_type: DatabaseType = DatabaseType.POSTGRESQL
    ) -> list[dict[str, str]]:
        """Build the prompt for OpenAI with
database metadata context.
```

```
        Args:
            user_prompt: Natural language query
from user
            metadata: Database schema metadata
dictionary
            db_type: Database type (PostgreSQL or
MySQL)

        Returns:
            List of messages for OpenAI chat
completion
        """
        # Build schema context
        schema_context = []
        for table in metadata.get("tables", []):
            columns_info = []
            for col in table.get("columns", []):
                col_desc = f"  - {col['name']}
({col['dataType']})"
                if col.get("primaryKey"):
                    col_desc += " PRIMARY KEY"
                if not col.get("nullable", True):
                    col_desc += " NOT NULL"
                if col.get("unique"):
                    col_desc += " UNIQUE"
                columns_info.append(col_desc)

            row_count = table.get("rowCount",
```

```
"unknown")
            table_info = f"Table:
{table['schemaName']}.{table['name']} ({row_count}
rows)\n"
            table_info += "\n".join(columns_info)
            schema_context.append(table_info)

        for view in metadata.get("views", []):
            columns_info = [f"  - {col['name']}
({col['dataType']})" for col in
view.get("columns", [])]
            view_info = f"View:
{view['schemaName']}.{view['name']}\n"
            view_info += "\n".join(columns_info)
            schema_context.append(view_info)

        schema_text = "\n\n".join(schema_context)

        # Build database-specific rules
        if db_type == DatabaseType.MYSQL:
            db_name = "MySQL"
            syntax_rules = """3. Use backticks for
identifiers (e.g., `table_name`, `column_name`)
4. Return valid MySQL syntax
5. Use MySQL LIMIT syntax (LIMIT n)
6. Be aware of MySQL-specific features like
AUTO_INCREMENT"""
        else:
            db_name = "PostgreSQL"
```

```
            syntax_rules = """3. Use proper schema
qualification (schema.table)
4. Return valid PostgreSQL syntax
5. Use double quotes for identifiers if needed"""

        system_message = f"""You are an expert SQL
query generator for {db_name} databases.

Database Schema:
{schema_text}

Rules:
1. Generate ONLY SELECT queries (no
INSERT/UPDATE/DELETE/DROP)
2. Always include LIMIT clause (max 1000 rows)
{syntax_rules}
7. Handle both English and Chinese natural
language
8. Be concise - return just the SQL query

Output format:
Return ONLY the SQL query, nothing else. No
explanations, no markdown, just the SQL."""

        return [
            {"role": "system", "content":
system_message},
            {"role": "user", "content":
user_prompt},
```

```
        ]

    async def generate_sql(
        self, user_prompt: str, metadata: dict,
db_type: DatabaseType = DatabaseType.POSTGRESQL
    ) -> dict[str, str]:
        """Convert natural language to SQL query.

        Args:
            user_prompt: Natural language query
            metadata: Database schema metadata
dictionary
            db_type: Database type (PostgreSQL or
MySQL)

        Returns:
            Dict with 'sql' and 'explanation' keys

        Raises:
            Exception: If OpenAI API call fails
        """
        try:
            messages =
self._build_prompt(user_prompt, metadata, db_type)

            # Call OpenAI API
            response = await
self.client.chat.completions.create(
                model=self.model,
```

```
                messages=messages,
                temperature=0.1,  # Low
temperature for consistent SQL generation
                max_tokens=500,
            )

            generated_sql =
response.choices[0].message.content.strip()

            # Clean up the response (remove
markdown code blocks if present)
            if generated_sql.startswith("```sql"):
                generated_sql =
generated_sql.replace("```sql", "").replace("```",
"").strip()
            elif generated_sql.startswith("```"):
                generated_sql =
generated_sql.replace("```", "").strip()

            # Generate explanation
            explanation = f"Generated SQL from:
{user_prompt}"

            logger.info(f"Generated SQL for
prompt: {user_prompt[:50]}...")

            return {"sql": generated_sql,
"explanation": explanation}
```

```
        except Exception as e:
            logger.error(f"Failed to generate SQL:
{str(e)}")
            raise Exception(f"Failed to generate
SQL: {str(e)}")


# Global instance
nl2sql_service = NaturalLanguageToSQLService()
```

来源与逐字稿导航

O 表示本次用户提供的原始课程资料与本地验证证据，不代表生财帖子。课程文件未披露的发布时间写未知；2026-10-29 是整理和快照日期，不是课程发布日期。

O1｜项目启动与配置

证据：`_raw/course/db_query/Makefile`。定位：Makefile全部目标；backend/pyproject.toml、frontend/package.json、app/config.py。 本地课程资料/验证日志，2026-10-29读取；用途与未测边界如上。

回到正文

O2｜w2五讲逐字稿

证据：`_raw/transcripts/L01/transcript.json`。定位：L01–L05，按原视频时间戳；自动转录。分课MD/SRT在deliver/transcripts。 本地课程资料/验证日志，2026-10-29读取；用途与未测边界如上。

回到正文

O3｜Cursor规则与命令

证据：`_raw/tooling/.cursor/rules/python-fastapi-backend.mdc`。定位：同目录规则及../commands下SpecKit等命令，完整原文已随templates附带。 本地课程资料/验证日志，2026-10-29读取；用途与未测边界如上。

回到正文

O4｜SpecKit项目产物

证据：`_raw/course/specs/001-db-query-tool/spec.md`。定位：同目录plan、tasks、research、data-model、quickstart、contracts/api-v1.yaml。 本地课程资料/验证日志，2026-10-29读取；用途与未测边界如上。

回到正文

O5｜后端真实调用链

证据：`_raw/course/db_query/backend/app/api/v1/databases.py`。定位：queries.py、services/sql_validator.py、query_wrapper.py、database_service.py、metadata.py、models、adapters。 本地课程资料/验证日志，2026-10-29读取；用途与未测边界如上。

回到正文

O6｜前端实际界面

证据：`_raw/course/db_query/frontend/src/pages/Home.tsx`。定位：components/下DatabaseSidebar、MetadataTree、SqlEditor、NaturalLanguageInput及services/api。 本地课程资料/验证日志，2026-10-29读取；用途与未测边界如上。

回到正文

O7｜NL2SQL原始实现

证据：`_raw/course/db_query/backend/app/services/nl2sql.py`。定位：全文件，_build_prompt与generate_sql；2026-10-29快照。 本地课程资料/验证日志，2026-10-29读取；用途与未测边界如上。

回到正文

O8｜MySQL扩展与代码评审

证据：`_raw/w3-db-extension/specs/db-query/0001-code-review.md`。定位：w3第4讲；.chats/w2六节、w3前两节、w1末两节；w2/specs/0001-improvement.md。 本地课程资料/验证日志，2026-10-29读取；用途与未测边界如上。

回到正文

O9｜隔离副本验证记录

证据：`practice-install.log`。另核`practice/backend-runtime-install.log`（运行依赖安装成功）、`practice/sql-boundary-results.json`（7例纯解析）、`practice/backend-unit-test.log`（64通过20失败4警告）、`practice/frontend-install.log`与`frontend-install-repair.log`（锁冲突与后续安装中止）。 本地课程资料/验证日志，2026-10-29读取；用途与未测边界如上。

回到正文

O10｜六讲逐字稿独立全文复核

2026-10-29，独立评审者逐段读取w2 L01–L05与w3 L04完整机器转录。证据：`_raw/transcripts/L01–L05/transcript.json`及`_raw/transcripts/w3-L04/transcript.json`。关键定位：w2第3讲00:17–00:27 SpecKit命令；第4讲00:03–00:42需求到任务；第5讲00:00–00:24分phase/安装/工具迁移，00:29–01:01 UI，01:31–01:40复杂NL错例，01:41–02:11分析/测试/作业；w3第4讲00:01–00:20扩展/测试/PG回归，00:21–00:31架构评审与重构指令。未以ASR中的技术错词确定API/版本；原机器稿不覆盖改写。

回到正文

O11｜屏幕输入与真实帧核对

原视频路径和秒数见`image-map.json`、`prompt-ledger.json`。9条新增完整可见输入、4条部分草稿分别登记；文件原prompt与屏幕variant不强行合并。图注写输入中/旧结果/失败测试等真实状态；图片提取与打开核看不等于本次应用试用。PDF图注同页及放大由导出后另核。

回到正文